

Tasks

- Declares a structure named Student to store:
 - First Name
 - Last Name
 - Roll Number
 - Marks
- Includes a member function displayStudentInfo() to:
 - Display the full name of the student.
 - Display the marks of the student.
- Uses a structure variable to:
 - Assign values to the data members.
 - Call the member function to display the student's information.

Tasks

- Repeat the previous program, but this time:
 - Use an array of structures to store details of multiple students.
 - Allow the user to input details for multiple students.
 - Call the displayStudentInfo() function for each student to display their full name and marks.
- Modify your previous program to:
 - Use a pointer to a structure to store and access student details.
 - Dynamically assign values to the structure members using the pointer.
 - Use the pointer to call the displayStudentInfo() function

Home Tasks

- What is the purpose of using **this** in member functions of structure?

Task 1: Student Record System (Functions Inside Class)

Create a class **Student** with the following attributes:

- ❑ string **name**
- ❑ int **rollNumber**
- ❑ float **marks**

Define the following member functions inside the class:

- ❑ **getData()** – to take input from the user
- ❑ **displayData()** – to display student information

Create an object and test the functionality.

Task 2: Rectangle(Functions Outside Class)

Create a class **Rectangle** with attributes:

- ❑ int **length**
- ❑ int **width**

Define the following member functions outside the class:

- ❑ **setDimensions()** – to input length and width
- ❑ **area()** – to calculate area
- ❑ **perimeter()** – to calculate perimeter

Create an object and test the functionality.

Task 3: Voting Eligibility

Create a class **Voter** with attributes:

- ❑ string **name**
- ❑ int **age**

Define the following member functions outside the class:

- ❑ **isEligible()** – to check the eligibility of the voter. (Only voter age > 18 is eligible). This method should return true or false based on the age of the person.
- ❑ Based on what the **isEligible()** method returns show an appropriate message to the user in main function.

Create an object and test the functionality.

Task 4: Temperature Convertor

Create a class named **Temperature** that converts temperature from Celsius to Fahrenheit. The class should have:

- ❑ A public data member to store temperature in Celsius.
- ❑ A function to convert the temperature using the formula.

$$\text{Fahrenheit} = (\text{Celsius} \times \frac{9}{5}) + 32$$

- ❑ A function to display the converted value.

Create an object and test the functionality.

Task 5: Calculator(Functions Outside Class)

Create a class **Calculator** with attributes:

- ❑ float **num1**
- ❑ float **num2**

Define the following member functions inside or outside the class:

- ❑ **add()**
- ❑ **subtract()**
- ❑ **multiply()**
- ❑ **divide()**

Allow the user to input two numbers and choose an operation to perform.

Task 5: Separate Header Files

Reorganize all the programs by creating separate header files to improve code structure and readability. Include these header files in the main program and demonstrate the use of objects accordingly.

Task 5: Encapsulation

Test Code:

Create a **Product** class with proper **encapsulation**.

Private data members:

- ❑ string **name**
- ❑ double **price**
- ❑ int **quantity**

Public Methods:

- ❑ void **setName(string n)** - Validate non-empty name.
- ❑ void **setPrice(double p)** - Validate price > 0.
- ❑ void **setQuantity(int q)** - Validate quantity >= 0
- ❑ **Getters** for all fields

```
int main() {  
    Product p;  
    p.setName(""); // Should print error message  
    p.setPrice(-50); // Should print error message  
    p.setQuantity(-2); // Should print error message  
  
    p.setName("Laptop");  
    p.setPrice(1200.50);  
    p.setQuantity(5);  
  
    cout << "Product: " << p.getName() << endl;  
    cout << "Price: $" << p.getPrice() << endl;  
    cout << "Quantity: " << p.getQuantity() << endl;  
    return 0;  
}
```

Task 1: Default Constructor — Employee Management System

Create a class named `Employee` for a basic employee management system. The class should have the following private data members:

- `id` (int)
- `name` (string)
- `salary` (float)

Implement a default constructor that initializes:

- `id` = 0
- `name` = "Not Assigned"
- `salary` = 0.0

Also include:

- A `displayDetails()` method that shows employee details.
- A `main()` function that creates an object using the default constructor and displays its information.

Objective: Simulate how a new employee might be stored in the system before their details are updated.

Prepared By: Engr. M. Ahsan 13

Task 2: Parameterized Constructor — Bank Account Initialization

Create a class named `BankAccount` that manages account information. It should have the following private data members::

- `accountNumber` (string)
- `accountHolder` (string)
- `balance` (double)

Implement a parameterized constructor to initialize all three values when a new account is created:

- `accountNumber` - PK0309800780100
- `accountHolder` - Muhammad Awaiz
- `balance` - 10000

Also include:

- A method `showAccountDetails()` to display the account information..
- In `main()`, create an object of `BankAccount` using user-defined values and display the account details.

Objective: Demonstrate how real banking systems initialize customer data upon account creation.

Prepared By: Engr. M. Ahsan 14

Task 3: Constructor Overloading

Create a class named `Rectangle` with the following private data members:

- `length` (float)
- `width` (float)

Overload the constructor as follows:

- A default constructor that initializes both length and width to 1.0.
- A parameterized constructor that takes two float values to initialize length and width.
- A single-parameter constructor that sets both length and width to the same value (creating a square).

Also include:

- A method `area()` to return the area of the rectangle and a `display()` method.
- In `main()`, create three objects using all constructor version dimensions and area.

14-16 / 21

Objective: Demonstrate how a various version of the constructors can be created using overloading.

Prepared By: Engr. M. Ahsan 15

Destructor in C++

- A special member function that is **automatically called when an object goes out of scope**.
- Used to **release resources** (memory, files, network, etc.).
- Name is **same as class**, prefixed with a tilde (~)

```
class MyClass {  
public:  
    ~MyClass() {  
        // Cleanup code here  
    }  
};
```

Key Points

- Called automatically for stack objects.
- Only one destructor per class (**no overloading**).
- No parameters, no return type.
- If not defined, C++ provides a default destructor

Page 16

16-17 / 21

When is Destructor Called?

- When the object goes out of scope (**stack**).
- When you explicitly use `delete` (**heap**)

```
{  
    MyClass obj; // destructor called automatically  
}  
MyClass* ptr = new MyClass();  
delete ptr; // destructor called manually
```

When Should You Define a Destructor?

- You allocate memory with `new`.
- You open files or use system/network resources.
- You want to track when objects are destroyed (e.g., for logging/debugging).

Prepared By: Engr. M. Ahsan 17

Practice Question

- What will be the output of the following code?

```
class MyClass {  
public:  
    MyClass() { cout << "Constructor()"; }  
    ~MyClass() { cout << "Destructor()"; }  
};  
  
int main() {  
    MyClass obj;  
    return 0;  
}
```

Output

Constructor
Destructor

17-18 / 21

Prepared By: Engr. M. Ahsan 18

Resource Cleanup Example

```
class Memory {  
private:  
    int* data;  
public:  
    Memory(int size) {  
        data = new int[size];  
        cout << "Memory of size " << size << " created.\n";  
    }  
  
    ~Memory() {  
        delete[] data;  
        cout << "Memory destroyed and memory released.\n";  
    }  
};
```

Prepared By: Engr. M. Ahsan 19

Task 4: Destructor

Create a class called **Locker** that represents a bank locker.

- The constructor should print:
 "Locker allocated to customer."
- The destructor should print:
 "Locker returned by customer."

Inside your `main()` function:

- Create **one** locker object inside a block { } to observe automatic destructor call.
- Create another locker using **new** and release it using **delete**.

Objective:

- Understand when destructors are called.
- Practice creating objects on stack and heap.
- Learn the importance of managing memory and cleanup.

Prepared By: Engr. M. Ahsan 20

Task – Single Inheritance

- You are building a student information system. Create a **Person** class and a **Student** class that inherits from it.
- **Classes & Requirements:**
 - **Person:**
Attributes: name, age
Functions: display_person_info()
 - **Student** (inherits Person):
Additional Attribute: student_id
Additional Function: display_student_info()
- **Tasks for Students:**
 - Define both classes using **single inheritance**.
 - Accept input from the user for all fields.
 - Display complete student information using both functions.

Task – Multilevel Inheritance

- You are designing a payroll system.
- **Classes & Requirements:**
 - **Person:**
Attributes: name, age
Function: display_person()
 - **Employee** (inherits Person):
Additional Attribute: employee_id
Additional Function: display_employee()
 - **Manager** (inherits Employee):
Additional Attribute: department
Function: display_manager()
- **Tasks for Students:**
 - Implement **multilevel inheritance**.
 - Accept input and display complete info using all display functions.

Task – Hierarchical Inheritance

- In an organization, Employee is a base class. Developer and Designer inherit from it.
- **Classes & Requirements:**
 - **Employee:**
Attributes: name, salary
Function: display_employee()
 - **Developer** (inherits Employee):
Additional Attribute: programming_language
Additional Function: display_developer()
 - **Designer** (inherits Employee):
Additional Attribute: design_tool
Function: display_designer()
- **Tasks for Students:**
 - Implement hierarchical inheritance.
 - Create objects of both derived classes and display all information.

Task – Multiple Inheritance

- Build a device that behaves as both a Printer and a Scanner.
- **Classes & Requirements:**
 - **Printer:**
Function: print_document ()
 - **Scanner**
Function: scan_document()
 - **Photocopier** (inherits from Printer and Scanner):
Function: photocopy() (calls both above functions)
- **Tasks for Students:**
 - Implement multiple inheritance.
 - Create an object of Photocopier and call all functions.

Object Oriented Programming (OOP)

“Lab 06”

(Constructors and Destructors in Inheritance)

Instructor: Engr.Awais Rathore

Course Code: CSC-521

Task 1: Online Shopping System (Constructor):

Create a base class **Product** with properties **name** and **price**. Create a derived class **Electronics** with property **warrantyYears**. Use constructor chaining to initialize all values and display them.

Task 2: Ride Booking System (Constructor - Multi-level):

Create classes **Vehicle** -> **Car** -> **ElectricCar**. Each class should have a constructor printing a message. Pass values through constructors and observe execution order.

Task 3: File Handling System (Destructor):

Create a base class **File** that prints '**File opened**' in constructor and '**File closed**' in destructor.

Create a derived class **TextFile** that prints its own messages. Create object and observe destructor order.

Task 4: Device and Sensor System (Destructor with Dynamic Memory):

Create base class **Device** with properties **deviceName**, **deviceId**, and dynamically allocated **data**.

Create derived class **Sensor** with **sensorType** and dynamically allocated **sensorValue**. Use constructors to allocate memory and destructors to free memory. Test using **new** and **delete**.

Composition Task

Design a simple **Calculator** class that can perform basic arithmetic operations such as addition and multiplication.

The calculator should display results using a separate class called **Display**.

The **Calculator** class must own a **Display** object as a part of its internal structure.

When the calculator performs a calculation, it should delegate the responsibility of showing the result to the **Display**.

The display should also keep track of the last result shown.

Prepared by : Engr. M.Awais

10

Aggregation Task

In a classroom, multiple students are solving math problems. However, instead of each student having their own calculator, the teacher provides one **shared calculator** that all students can use.

Each student can:

- Use the calculator to perform addition or multiplication.
- Request the result of an operation by calling the calculator.

The **calculator is not owned by any student**. It is created and managed externally (by the teacher or system), and students just refer to it.

Prepared by : Engr. M.Awais

11

Composition

```
cpp
#include <iostream>

class Display {
    double lastValue;
public:
    Display() {
        lastValue = 0.0;
    }

    void show(double value) {
        lastValue = value;
        std::cout << "Result: " << lastValue << "\n";
    }

    double getLastValue() const {
        return lastValue;
    }
};

class Calculator {
    Display screen; // COMPOSITION: screen is a part of Calculator
public:
    Calculator() {}

    void add(double a, double b) {
        double result = a + b;
        screen.show(result);
    }

    void multiply(double a, double b) {
        double result = a * b;
        screen.show(result);
    }

    void showPreviousResult() const {
        std::cout << "Previous result: " << screen.getLastValue() << "\n";
    }
};

int main() {
    Calculator calc;

    calc.add(10, 5); // Output: 15
    calc.multiply(3, 7); // Output: 21
    calc.showPreviousResult(); // Output: 21

    return 0;
}
```

Prepared by : Engr. M.Awais

12

Aggregation

```
#include <iostream>

class Calculator {
public:
    double add(double a, double b) {
        return a + b;
    }

    double multiply(double a, double b) {
        return a * b;
    }
};

class Student {
    Calculator* calc; // Aggregation: uses an external calculator
public:
    Student(Calculator* c) {
        calc = c;
    }

    void solveAddition(double a, double b) {
        if (calc) {
            std::cout << "Addition result: " << calc->add(a, b) << "\n";
        }
    }

    void solveMultiplication(double a, double b) {
        if (calc) {
            std::cout << "Multiplication result: "

```